Composants Modulaires, Props, État Local et Immutabilité

Objectifs

Au cours de cet atelier pratique, vous allez :

1. Découper une interface monolithique en une arborescence de composants fonctionnels réutilisables.
2. Transmettre des données typées d'un composant parent vers un composant enfant en utilisant les `props`.
3. Exploiter la prop spéciale `children` pour concevoir des composants conteneurs génériques.
4. Gérer l'état interne réactif d'un composant à l'aide du Hook `useState`.
5. Mettre en œuvre les règles strictes d'immutabilité pour la mise à jour d'objets et de tableaux dans l'état.
6. Implémenter le principe de remontée d'état (*Lifting State Up*) pour faire communiquer un enfant avec son parent via des fonctions de rappel (*callbacks*).

Contexte du Projet : OmniMonitor - Tableau de Bord Opérationnel

Vous poursuivez le développement de l'application de supervision **OmniMonitor**. L'objectif de cet atelier est de transformer la vue statique initiale en un tableau de bord dynamique et interactif permettant de surveiller en temps réel un parc de stations météorologiques, d'activer ou de désactiver des stations, de filtrer les données et d'ajouter de nouvelles stations à la volée.

Étape 1 : Structuration de l'Arborescence des Composants

Pour respecter le principe de responsabilité unique, vous allez organiser vos fichiers sources dans un dossier `src/components/`.

1. Créez un dossier `src/components/` s'il n'existe pas.
2. Créez un composant d'en-tête `src/components/Header.jsx` qui affiche le titre du portail et un sous-titre de description.
3. Créez un composant de carte individuelle `src/components/StationCard.jsx` qui affichera les informations d'une station.

4. Organisez votre composant principal `src/App.jsx` pour qu'il importe et assemble ces composants.

Étape 2 : Transmission et Typage des `Props`

Dans cette étape, vous allez configurer le composant `StationCard` pour qu'il accepte des données transmises par son parent.

1. Dans `src/App.jsx`, définissez un tableau d'objets représentant les stations d'observation :
    - Station 1 : `id: 1`, `name: 'Toulouse-Blagnac'`, `altitude: 152`, `status: 'online'`, `sensorsCount: 6`
    - Station 2 : `id: 2`, `name: 'Pic du Midi'`, `altitude: 2877`, `status: 'offline'`, `sensorsCount: 3`
    - Station 3 : `id: 3`, `name: 'Marseille-Marignane'`, `altitude: 22`, `status: 'online'`, `sensorsCount: 5`
2. Modifiez le composant `src/components/StationCard.jsx` pour qu'il déstructure ses `props` directement dans la signature de la fonction :
    - `name`
    - `altitude` (avec une valeur par défaut de `0`)
    - `status` (avec une valeur par défaut de `'online'`)
    - `sensorsCount`
3. Affichez ces informations proprement dans une balise `<article className="card">`. Utilisez une condition pour afficher un badge vert (`badge-online`) si le statut est `'online'`, ou rouge (`badge-offline`) s'il est `'offline'`.
4. Dans `src/App.jsx`, utilisez la méthode `.map()` pour parcourir le tableau de stations et générer dynamiquement une liste de composants `<StationCard />` (n'oubliez pas d'ajouter l'attribut `key={station.id}` sur chaque carte).

Étape 3 : Utilisation de la Prop Spéciale `children`

Vous allez créer un composant conteneur générique pour encadrer les blocs d'alerte.

1. Créez un fichier `src/components/Panel.jsx`.
2. Ce composant doit accepter deux props : `title` et `children`.
3. Il retourne une structure HTML avec un en-tête contenant le titre et un corps de texte affichant dynamiquement `{children}`.

4. Intégrez ce composant `Panel` dans `src/App.jsx` pour y envelopper un message d'alerte contextuel.

Étape 4 : Gestion de l'État Local (`useState`)

Vous allez ajouter de l'interactivité au niveau d'une station individuelle ou du tableau de bord.

1. Dans un nouveau composant ou directement dans `StationCard`, importez le Hook `useState` depuis `'react'`.
2. Ajoutez un état local booléen `isExpanded` (initialisé à `false`) qui permet d'afficher ou de masquer les détails techniques de la station au clic sur un bouton "Afficher les détails".
3. Testez le comportement : le clic modifie l'état local et déclenche le re-rendu ciblé de la carte concernée.

Étape 5 : Immutabilité et Modification d'un Tableau dans l'État

Vous allez maintenant permettre d'ajouter une nouvelle station dynamique et de supprimer une station existante.

1. Dans `src/App.jsx`, déplacez le tableau des stations initiales dans un état local :

   ```jsx
   const [stations, setStations] = useState([
     { id: 1, name: 'Toulouse-Blagnac', altitude: 152, status: 'online', sensorsCount: 6 },
     { id: 2, name: 'Pic du Midi', altitude: 2877, status: 'offline', sensorsCount: 3 },
     { id: 3, name: 'Marseille-Marignane', altitude: 22, status: 'online', sensorsCount: 5 }
   ]);
   ```

2. Créez une fonction `handleAddStation` qui génère un nouvel objet station et utilise la syntaxe immuable de mise à jour du tableau :

   ```jsx
   setStations(prevStations => [...prevStations, newStation]);
   ```

   Attention : N'utilisez jamais `stations.push()`.

3. Créez une fonction `handleRemoveStation(id)` qui utilise `.filter()` pour retirer une station du tableau d'état :

```jsx
setStations(prevStations => prevStations.filter(st => st.id !== id));
```

4. Ajoutez un bouton dans l'interface pour déclencher l'ajout d'une station fictive et un bouton sur chaque carte pour supprimer la station correspondante.

Étape 6 : Remontée d'État (Lifting State Up)

Pour qu'un bouton situé à l'intérieur de la carte enfant (`StationCard`) puisse demander au composant parent (`App`) de supprimer la station, vous devez mettre en place une communication par callback.

1. Le composant parent `App` définit la logique de suppression (`handleRemoveStation`).
2. Le parent transmet cette fonction à l'enfant via une prop nommée `onDelete` :

```jsx
<StationCard
  key={st.id}
  {...st}
  onDelete={handleRemoveStation}
/>
```

3. Dans `StationCard`, le bouton de suppression appelle la fonction reçue en argument :

```jsx
<button onClick={() => onDelete(id)} className="btn-danger">
  Supprimer
</button>
```

Critères de Validation

Avant de valider ce segment, vérifiez que :

- L'arborescence des composants est bien séparée dans `src/components/`.
- Aucune modification directe d'une prop ou mutation d'un tableau avec `.push()` n'est présente dans le code.

- La suppression et l'ajout de stations mettent à jour l'interface de manière fluide et réactive.
- Le flux de données respecte strictement le sens descendant (`props`) et ascendant (`callbacks`).